MACHINE LEARNING · LAB ML-08

Deep Convolutional Neural Network (DCNN) กับการจำแนกประเภทสัตว์ด้วยชุดข้อมูล Zoo

เปรียบเทียบจำนวน epoch และโครงสร้างของ DCNN แบบ VGG (จำนวนชั้น convolution และจำนวน neuron) แล้ววัดผลด้วยค่า accuracy

101
ตัวอย่าง (สัตว์) · 16 features · 7 คลาส

5 + 9
ค่า epoch ที่ทดลอง + โครงสร้าง DCNN ที่เปรียบเทียบ

100.00%
Test accuracy ของโมเดลสุดท้าย (16 ตัวอย่าง)

จัดทำโดย จักรพัทศ์ ยอดระยับ
สาขาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี (RMUTT)
เครื่องมือ: Python · TensorFlow/Keras · scikit-learn · matplotlib

1. บทนำและวัตถุประสงค์

Deep Convolutional Neural Network (DCNN) คือโครงข่ายประสาทเทียมแบบ convolution ที่ซ้อนชั้น convolution หลายชั้นต่อกัน เพื่อให้โมเดลเรียนรู้ลักษณะเด่น (feature) จากระดับง่ายไปสู่ระดับซับซ้อนได้เอง ตัวอย่างที่รู้จักกันดีคือ VGG ซึ่งใช้ตัวกรองขนาดเล็ก (ความกว้าง 3) ซ้อนกันเป็นบล็อก บล็อกละ 2 ชั้น แล้วเพิ่มจำนวนตัวกรองเป็นสองเท่าหลังการ pooling ทุกครั้ง

รายงานนี้นำแนวคิดดังกล่าวมาใช้จำแนกประเภทสัตว์ 7 กลุ่มจากชุดข้อมูล Zoo ตามโจทย์ LAB ซึ่งกำหนดให้

- เลือกและโหลดชุดข้อมูล แบ่งเป็นชุด train / test และทำ standardization ก่อนฝึกโมเดล
- สร้างโมเดล DCNN และฝึกด้วยจำนวน epoch ที่ต่างกัน
- เปรียบเทียบโครงสร้าง DCNN หลายแบบ (จำนวนชั้น convolution และจำนวน neuron) แล้ววัดผลด้วย accuracy
- แสดงกราฟ accuracy/loss ของ training และ validation พร้อมผลการทำนาย

> **หมายเหตุเรื่องชนิดข้อมูล:** โครงสร้างโปรเจกต์ต้นแบบออกแบบไว้สำหรับรูปภาพ (โฟลเดอร์ PetImages และ VGG แบบ 2 มิติ) แต่ชุดข้อมูลที่ใช้ในงานนี้เป็นข้อมูลตาราง จึงปรับเป็น **VGG แบบ 1 มิติ** (Conv1D / MaxPooling1D) โดยมองคุณลักษณะ 16 ค่าของสัตว์แต่ละตัวเป็นลำดับความยาว 16 จำนวน 1 channel ชื่อไฟล์และขั้นตอนทั้งหมดยังคงตามโครงสร้างเดิม ยกเว้นโฟลเดอร์ PetImages ที่ไม่จำเป็นต้องใช้

2. ชุดข้อมูล (dataset.csv)

ชุดข้อมูล Zoo (UCI Machine Learning Repository) มีสัตว์ 101 ตัวอย่าง แต่ละตัวอย่างมี 16 features และป้ายกำกับ 1 ค่า คือประเภทสัตว์ 7 กลุ่ม ไฟล์ `dataset.csv` ได้จากการรวมไฟล์ `zoo.csv` (ข้อมูลหลัก) กับ `class.csv` (ชื่อคลาส) ที่แนบมา โดยเพิ่มคอลัมน์ `class_name` ต่อท้าย

- **Features แบบ 0/1 จำนวน 15 ค่า:** hair, feathers, eggs, milk, airborne, aquatic, predator, toothed, backbone, breathes, venomous, fins, tail, domestic, catsize
- **Feature เชิงตัวเลข 1 ค่า:** legs (จำนวนขา มีค่า 0, 2, 4, 5, 6, 8)
- **ป้ายกำกับ:** class_type (1–7) ซึ่งในโค้ดแปลงเป็นดัชนี 0–6

รหัส	คลาส	ความหมาย	ทั้งหมด	Train	Val	Test	ตัวอย่างสัตว์
0	Mammal	สัตว์เลี้ยงลูกด้วยนม	41	28	7	6	aardvark, antelope, bear
1	Bird	นก	20	14	3	3	chicken, crow, dove
2	Reptile	สัตว์เลื้อยคลาน	5	3	1	1	pitviper, seasnake, slowworm
3	Fish	ปลา	13	9	2	2	bass, carp, catfish
4	Amphibian	สัตว์สะเทินน้ำสะเทินบก	4	2	1	1	frog, frog, newt
5	Bug	แมลง	8	6	1	1	flea, gnat, honeybee
6	Invertebrate	สัตว์ไม่มีกระดูกสันหลัง	10	7	1	2	clam, crab, crayfish
	รวม		**101**	**69**	**16**	**16**	

ตารางที่ 1 จำนวนตัวอย่างในแต่ละคลาส และการกระจายตัวหลังแบ่งข้อมูล

ข้อสังเกตสำคัญคือชุดข้อมูล**เล็กและไม่สมดุล**: Mammal มี 41 ตัวอย่าง แต่ Amphibian มีเพียง 4 และ Reptile มี 5 ข้อเท็จจริงนี้มีผลต่อการออกแบบการทดลองและการตีความผลในหัวข้อถัดไป

3. ขั้นตอนการทำงาน

ไฟล์ `classification/main.py` เรียกโมดูลย่อยตามลำดับ 6 ขั้นตอนดังนี้

ขั้นที่ 1 โหลดข้อมูล (data_loader.py)

อ่าน `dataset.csv` แล้วข้ามแถวที่เสียหาย (ค่าว่าง ค่าที่ไม่ใช่ตัวเลข หรือรหัสคลาสนอกช่วง 1–7) ซึ่งเทียบได้กับการข้ามไฟล์รูปที่เสียในโปรเจกต์ต้นแบบ ผลคือโหลดได้ครบ 101 แถว ไม่มีแถวที่ต้องข้าม

ขั้นที่ 2 แบ่งข้อมูล (split_data.py)

แบ่งแบบ stratified เป็น train / validation / test = 69 / 16 / 16 ตัวอย่าง (ประมาณ 70 / 15 / 15) การแบ่งแบบ stratified ทำให้ทุกคลาสปรากฏครบทั้งสามชุด แม้แต่คลาสที่มีเพียง 4 ตัวอย่าง (ดูตารางที่ 1) ชุด validation ใช้เลือกค่า epoch และโครงสร้าง ส่วนชุด test เก็บไว้วัดผลสุดท้ายเท่านั้น

ขั้นที่ 3 Standardization (preprocessing.py)

ปรับทุก feature ให้มีค่าเฉลี่ย 0 และส่วนเบี่ยงเบนมาตรฐาน 1 ด้วยสูตร $z = (x - \mu) / \sigma$ เหตุผลคือ legs มีค่าได้ถึง 8 ขณะที่ feature อื่นเป็น 0/1 หากไม่ปรับ feature ที่มีสเกลใหญ่จะมีอิทธิพลต่อ gradient มากเกินไป ค่า μ และ σ คำนวณจาก**ชุด train เท่านั้น** แล้วนำไปใช้กับ validation และ test เพื่อไม่ให้ข้อมูลจากชุดทดสอบรั่วเข้าสู่การฝึก (data leakage) จากนั้นแปลงรูปร่างจาก (n, 16) เป็น (n, 16, 1) ให้ตรงกับ input ของ Conv1D

ขั้นที่ 4 สร้างโมเดล DCNN (vgg_model.py)

ฟังก์ชัน `build_vgg_model()` สร้างโมเดลแบบ VGG โดยรับพารามิเตอร์ 2 ตัวที่ใช้เปรียบเทียบ ได้แก่ `conv_layers` (จำนวนชั้น convolution: 2, 4 หรือ 6 ชั้น คือ 1–3 บล็อก) และ `dense_units` (จำนวน neuron ในชั้น fully-connected) จำนวนตัวกรองเริ่มที่ 32 และเพิ่มเป็น 64 และ 128 ในบล็อกถัดไป ตารางที่ 2 แสดงโครงสร้างของโมเดลที่ถูกเลือกเป็นโมเดลสุดท้าย

ชื่อเลเยอร์	ชนิด	Output shape	พารามิเตอร์
`block1_conv1`	Conv1D(32, kernel=3, same, ReLU)	(16, 32)	128
`block1_conv2`	Conv1D(32, kernel=3, same, ReLU)	(16, 32)	3,104
`block1_pool`	MaxPooling1D(2)	(8, 32)	0
`block2_conv1`	Conv1D(64, kernel=3, same, ReLU)	(8, 64)	6,208
`block2_conv2`	Conv1D(64, kernel=3, same, ReLU)	(8, 64)	12,352
`block2_pool`	MaxPooling1D(2)	(4, 64)	0
`flatten`	Flatten	(256)	0
`fc`	Dense(32, ReLU)	(32)	8,224
`dropout`	Dropout(0.3)	(32)	0
`predictions`	Dense(7, Softmax)	(7)	231
รวม			**30,247**

ตารางที่ 2 โครงสร้างโมเดลสุดท้าย (4 ชั้น convolution, 32 neurons) input shape = (16, 1)

ขั้นที่ 5 การฝึกและการออกแบบการทดลอง

- **การฝึก:** optimizer Adam (learning rate 0.001), loss แบบ sparse categorical cross-entropy, batch size 16
- **การทดลองที่ 1:** ตรึงโครงสร้างไว้ (4 conv, 64 neurons) แล้วเปลี่ยนจำนวน epoch เป็น 10, 25, 50, 100, 200
- **การทดลองที่ 2:** ตรึงจำนวน epoch ที่ดีที่สุดจากการทดลองที่ 1 แล้วเปรียบเทียบ 3 × 3 = 9 โครงสร้าง
- **ความน่าเชื่อถือ:** ทุกการตั้งค่าฝึกซ้ำ 3 รอบด้วย random seed ต่างกัน แล้วรายงานค่าเฉลี่ย เพราะชุด test มีเพียง 16 ตัวอย่าง การทายผิด 1 ตัวอย่างจึงเปลี่ยน accuracy ถึง 6.25%
- **เกณฑ์การเลือก:** เลือกจาก validation accuracy เฉลี่ยสูงสุด ถ้าเท่ากันเลือกค่าที่ validation loss ต่ำกว่า โดยไม่ใช้ชุด test ในการตัดสินใจ

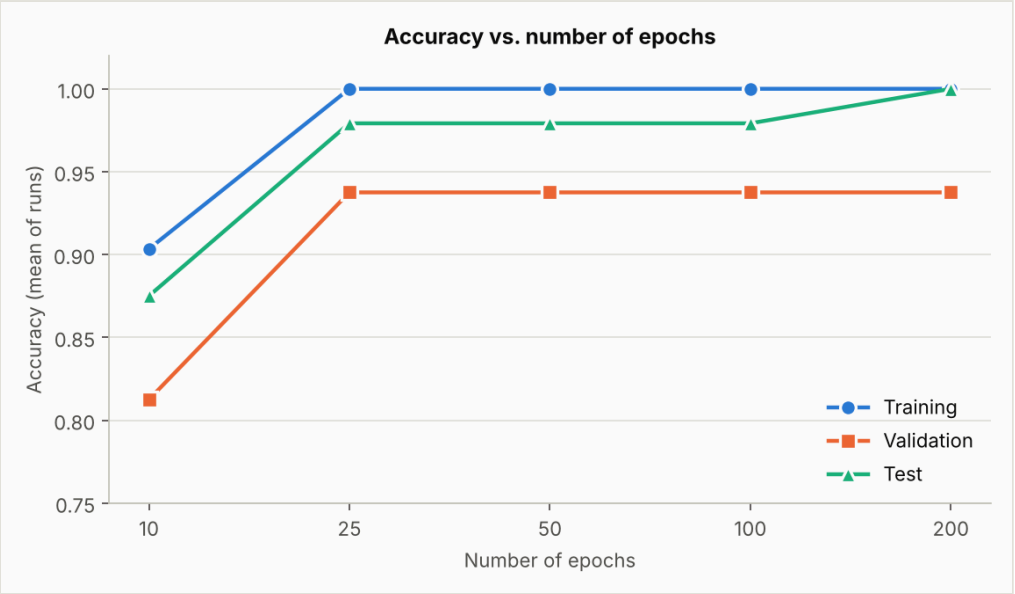
ขั้นที่ 6 ประเมินผล (evaluate.py, test_vgg.py)

วัด accuracy, classification report และ confusion matrix บนชุด test พร้อมบันทึกกราฟ training history จากนั้น `test_vgg.py` โหลดโมเดลที่บันทึกไว้มาทำนายตัวอย่างสุ่ม 4 ตัว

4. การทดลองที่ 1: จำนวน epoch

Epochs	Train acc	Val acc	Val loss	Test acc (เฉลี่ย ± SD)	Test acc แต่ละรอบ (seed 0 / 1 / 2)
10	0.903	0.812	0.524	0.875 ± 0.000	0.8750 / 0.8750 / 0.8750
25	1.000	0.938	0.259	0.979 ± 0.029	1.0000 / 0.9375 / 1.0000
50	**1.000**	**0.938**	**0.184**	**0.979 ± 0.029**	**1.0000 / 0.9375 / 1.0000**
100	1.000	0.938	0.293	0.979 ± 0.029	1.0000 / 0.9375 / 1.0000
200	1.000	0.938	0.370	1.000 ± 0.000	1.0000 / 1.0000 / 1.0000

ตารางที่ 3 ผลของจำนวน epoch (โครงสร้าง 4 conv / 64 neurons, เฉลี่ย 3 รอบ) แถวที่เน้นคือค่าที่ถูกเลือก



รูปที่ 1 Accuracy เฉลี่ยของชุด train / validation / test เมื่อเปลี่ยนจำนวน epoch (แกนตั้งขยายช่วง 0.75–1.00)

สิ่งที่เห็นจากผลการทดลอง

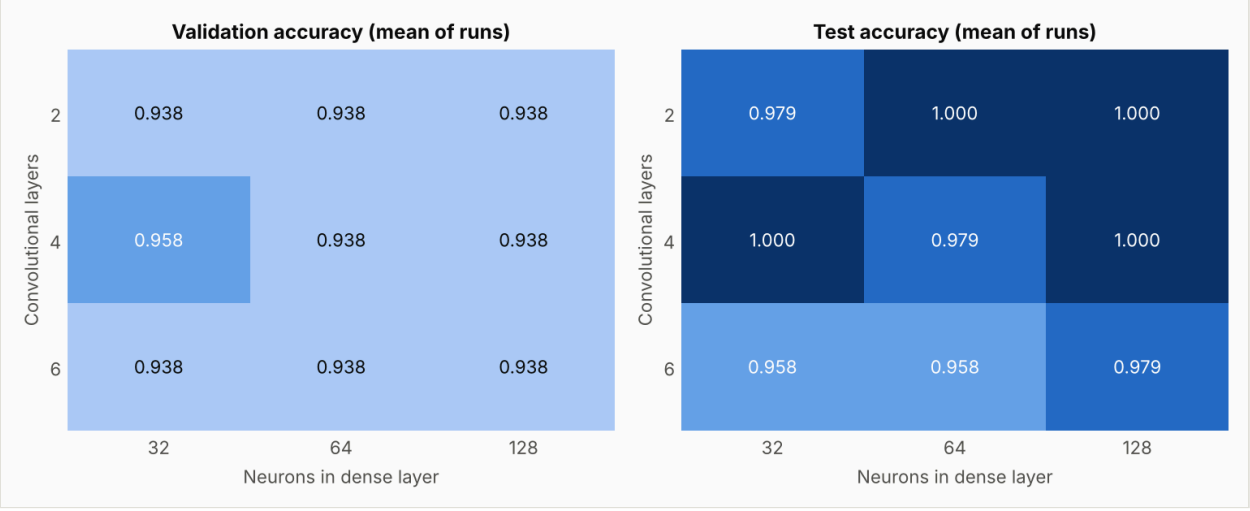
- **10 epochs ยังเรียนรู้ไม่พอ (underfitting):** train accuracy ได้เพียง 0.903 และ test accuracy 0.875 ต่ำกว่าทุกค่าที่เหลือ
- **ตั้งแต่ 25 epochs ขึ้นไป accuracy อิ่มตัว:** train accuracy ถึง 1.000 และ validation accuracy คงที่ที่ 0.938 ทุกค่า
- **ฝึกนานเกินเริ่มมีสัญญาณ overfitting:** validation loss ต่ำสุดที่ 50 epochs (0.184) แล้วสูงขึ้นเป็น 0.293 ที่ 100 epochs และ 0.370 ที่ 200 epochs แม้ accuracy ไม่ลดลง แปลว่าโมเดลมั่นใจกับคำตอบที่ผิดมากขึ้น
- **200 epochs ได้ test accuracy 1.000** สูงกว่า 25–100 epochs (0.979) แต่ส่วนต่างนี้มาจาก ตัวอย่างเพียง 1 ตัวใน 1 จาก 3 รอบ จึงยังสรุปไม่ได้ว่าดีกว่าจริง

จึงเลือก **50 epochs** ตามเกณฑ์ (validation accuracy เท่ากัน แต่ validation loss ต่ำสุด) ไปใช้ในการทดลองที่ 2

5. การทดลองที่ 2: โครงสร้าง DCNN

Conv layers	VGG blocks	Dense neurons	พารามิเตอร์	Train acc	Val acc	Val loss	Test acc (เฉลี่ย ± SD)
2	1	32	11,687	1.000	0.938	0.259	0.979 ± 0.029
2	1	64	20,135	1.000	0.938	0.224	1.000 ± 0.000
2	1	128	37,031	1.000	0.938	0.280	1.000 ± 0.000
4	**2**	**32**	**30,247**	**1.000**	**0.958**	**0.268**	**1.000 ± 0.000**
4	2	64	38,695	1.000	0.938	0.184	0.979 ± 0.029
4	2	128	55,591	1.000	0.938	0.167	1.000 ± 0.000
6	3	32	104,231	1.000	0.938	0.385	0.958 ± 0.029
6	3	64	112,679	1.000	0.938	0.541	0.958 ± 0.029
6	3	128	129,575	1.000	0.938	0.537	0.979 ± 0.029

ตารางที่ 4 Accuracy ของ DCNN ทั้ง 9 โครงสร้าง (ฝึก 50 epochs, เฉลี่ย 3 รอบ) แถวที่เน้นคือโครงสร้างที่ถูกเลือก



รูปที่ 2 Validation และ test accuracy เฉลี่ยของแต่ละโครงสร้าง (สีเข้ม = accuracy สูง)

สิ่งที่เห็นจากผลการทดลอง

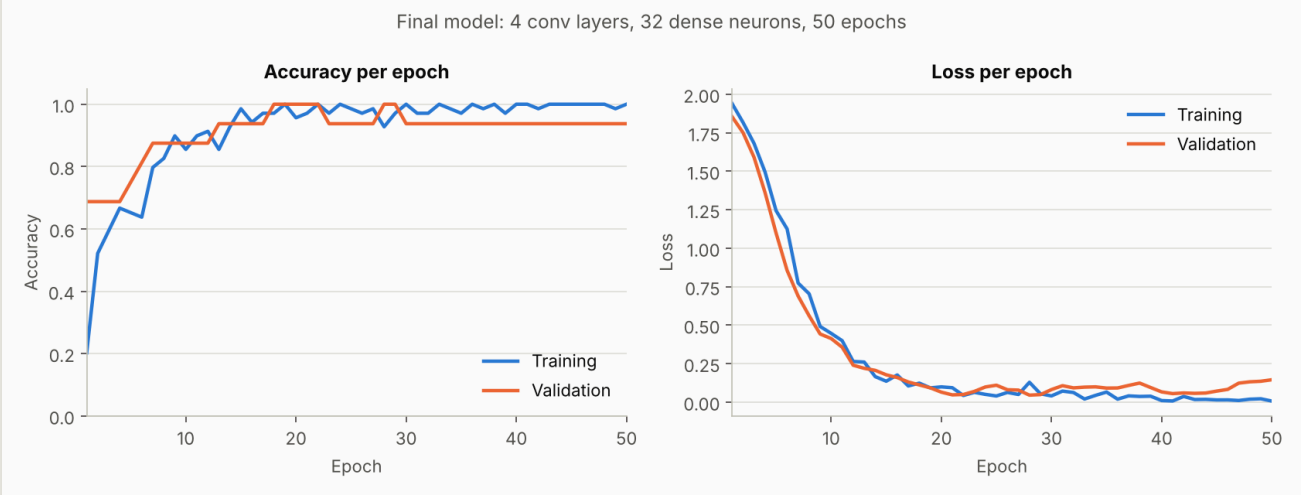
- **ทุกโครงสร้างจำชุด train ได้ครบ** (train accuracy 1.000) และได้ test accuracy เฉลี่ยตั้งแต่ 0.958 ถึง 1.000 ซึ่งต่างกันไม่เกิน 1 ตัวอย่างต่อรอบ
- **โมเดลที่ลึกที่สุดไม่ได้ดีที่สุด:** 6 ชั้น convolution ให้ test accuracy เฉลี่ย 0.965 ต่ำกว่า 2 ชั้น (0.993) และ 4 ชั้น (0.993) ทั้งยังมี validation loss เฉลี่ยสูงสุด (0.488 เทียบกับ 0.255 และ 0.206) โมเดลกลุ่มนี้มีพารามิเตอร์กว่า 100,000 ตัว ขณะที่ชุด train มีเพียง 69 ตัวอย่าง จึง overfit ได้ง่าย
- **จำนวน neuron ไม่แสดงแนวโน้มที่ชัดเจน:** การเพิ่มจาก 32 เป็น 128 ไม่ได้ทำให้ accuracy ดีขึ้นอย่างสม่ำเสมอ ความแตกต่างที่เห็นอยู่ในระดับเดียวกับความแปรปรวนระหว่างรอบ (SD 0.029)
- **โครงสร้างที่ถูกเลือก:** 4 ชั้น convolution กับ 32 neurons เพราะเป็นโครงสร้างเดียวที่ validation accuracy เฉลี่ยสูงกว่าที่เหลือ (0.958 เทียบกับ 0.938) และได้ test accuracy 1.000 ทั้ง 3 รอบ

6. โมเดลสุดท้ายและผลการทำนาย

ฝึกโมเดลสุดท้ายด้วยโครงสร้าง 4 ชั้น convolution, 32 neurons, 50 epochs (seed 42) แล้วบันทึกเป็น `outputs/vgg_model.keras` ได้ผลดังนี้

ชุดข้อมูล	จำนวนตัวอย่าง	Accuracy
Training	69	100.00%
Validation	16	93.75% (ผิด 1 ตัวอย่าง)
Test	**16**	**100.00%**

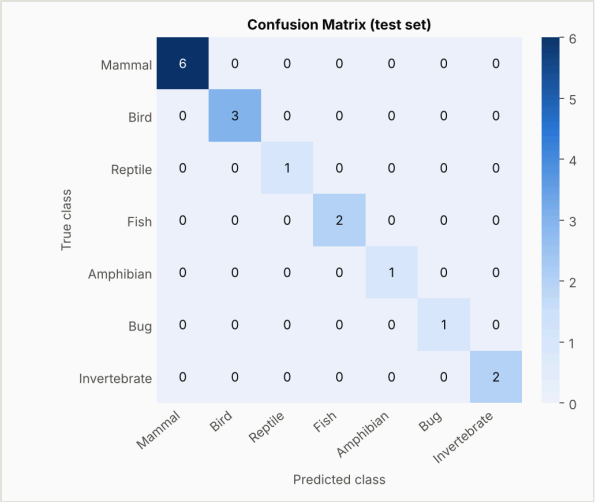
ตารางที่ 5 Accuracy ของโมเดลสุดท้าย



รูปที่ 3 Training และ validation accuracy/loss ต่อ epoch ของโมเดลสุดท้าย

กราฟแสดงว่า loss ลดลงเร็วในช่วง 10 epochs แรก และ accuracy ของทั้งสองชุดขึ้นไปเกิน 0.9 ภายในราว 15 epochs validation loss ต่ำสุดที่ epoch 28 (0.045) แล้วค่อย ๆ สูงขึ้นเป็น 0.145 ที่ epoch สุดท้าย ขณะที่ training loss ลดลงต่อเนื่องจนเหลือ 0.007 ช่องว่างที่เริ่มถ่างออกนี้เป็นสัญญาณ overfitting ระดับเล็กน้อย

ตัวอย่างเดียวที่โมเดลทายผิดในชุด validation คือ **seasnake** (งูทะเล) ซึ่งเป็น Reptile แต่ถูกทายเป็น Fish เมื่อดูข้อมูลจะเห็นเหตุผล: งูทะเลอาศัยในน้ำ ไม่มีขา และในชุดข้อมูลระบุว่าไม่ได้หายใจด้วยอากาศและไม่วางไข่ ซึ่งต่างจาก Reptile ตัวอื่นและคล้าย Fish มาก อีกทั้งชุด train มี Reptile ให้เรียนรู้เพียง 3 ตัวอย่าง



รูปที่ 4 Confusion matrix บนชุด test
ทุกตัวอย่างอยู่บนเส้นทแยงมุม (ทายถูกทั้งหมด)

```
Test accuracy: 1.0000

              precision    recall  f1-score   support

      Mammal       1.00      1.00      1.00         6
        Bird       1.00      1.00      1.00         3
     Reptile       1.00      1.00      1.00         1
        Fish       1.00      1.00      1.00         2
   Amphibian       1.00      1.00      1.00         1
         Bug       1.00      1.00      1.00         1
Invertebrate       1.00      1.00      1.00         2

    accuracy                           1.00        16
   macro avg       1.00      1.00      1.00        16
weighted avg       1.00      1.00      1.00        16
```

ตารางที่ 6 Classification report บนชุด test
(จาก `outputs/classification_report.txt`)

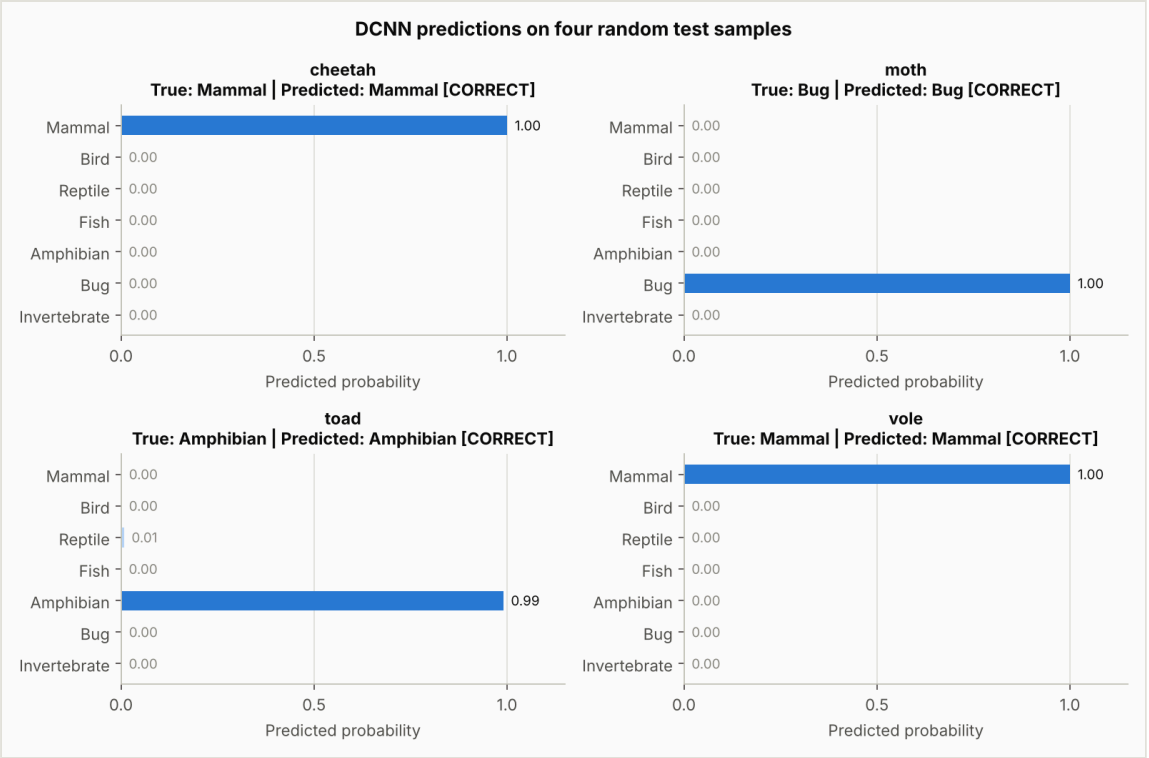
ผลการทำนายบนชุด test

สัตว์	คลาสจริง	คลาสที่ทำนาย	ความมั่นใจ
octopus	Invertebrate	Invertebrate	0.9985
slowworm	Reptile	Reptile	0.9738
carp	Fish	Fish	1.0000
lynx	Mammal	Mammal	1.0000
crab	Invertebrate	Invertebrate	0.9999
leopard	Mammal	Mammal	1.0000
pony	Mammal	Mammal	1.0000
pike	Fish	Fish	1.0000

สัตว์	คลาสจริง	คลาสที่ทำนาย	ความมั่นใจ
moth	Bug	Bug	1.0000
hawk	Bird	Bird	1.0000
vole	Mammal	Mammal	1.0000
swan	Bird	Bird	1.0000
cheetah	Mammal	Mammal	1.0000
lark	Bird	Bird	1.0000
toad	Amphibian	Amphibian	0.9900
goat	Mammal	Mammal	1.0000

ตารางที่ 7 ผลการทำนายทั้ง 16 ตัวอย่างของชุด test (ทายถูกทั้งหมด; จาก `outputs/predictions.csv`)

โมเดลทายถูกครบทั้ง 16 ตัวอย่าง ตัวอย่างที่ความมั่นใจต่ำที่สุดคือ slowworm (Reptile, 0.9738) ซึ่งอยู่ในคลาสที่มีข้อมูลน้อย



รูปที่ 5 ผลจาก `test_vgg.py --seed 7` : ความน่าจะเป็นของแต่ละคลาสสำหรับตัวอย่างสุ่ม 4 ตัว

7. อภิปรายผลและข้อจำกัด

- **ชุด test เล็กมาก:** 16 ตัวอย่างทำให้ accuracy ขยับทีละ 6.25% ค่า 100% จึงหมายถึง “ถูก 16 จาก 16” ไม่ได้รับประกันว่าจะถูกทุกครั้งกับสัตว์ชนิดใหม่ ผลบน validation (ผิด 1 จาก 16) สะท้อนภาพที่ระมัดระวังกว่า
- **คลาสเล็กมีตัวอย่างทดสอบคลาสละ 1 ตัว:** Reptile, Amphibian และ Bug มีเพียง 1 ตัวอย่างในชุด test ค่า precision/recall ของคลาสเหล่านี้จึงยังไม่มีน้ำหนักทางสถิติ
- **ความแตกต่างระหว่างโครงสร้างอยู่ในระดับ noise:** ส่วนต่างส่วนใหญ่เท่ากับ 1 ตัวอย่างใน 1 รอบ ข้อสรุปที่มั่นใจได้มีเพียงว่า 10 epochs น้อยเกินไป และโมเดล 6 ชั้นไม่ได้ให้ประโยชน์เพิ่ม
- **ข้อมูลตารางไม่มีลำดับเชิงพื้นที่:** Conv1D ถือว่า feature ที่อยู่ติดกันมีความสัมพันธ์กัน (เช่น hair–feathers–eggs) แต่ลำดับคอลัมน์ในตารางเป็นเพียงลำดับที่ผู้จัดทำชุดข้อมูลกำหนด DCNN จึงทำงานได้ แต่ไม่ได้เปรียบเหมือนที่ได้เปรียบกับรูปภาพ

แนวทางพัฒนาต่อ: ใช้ stratified k-fold cross-validation เพื่อให้ทุกตัวอย่างได้เป็นข้อมูลทดสอบ, เพิ่ม EarlyStopping โดยดู validation loss แทนการกำหนด epoch ตายตัว, ใช้ class weight ชดเชยคลาสที่มีข้อมูลน้อย และเปรียบเทียบกับโมเดลพื้นฐาน เช่น Nearest Neighbor หรือ MLP บนข้อมูลชุดเดียวกัน

8. สรุป

- สร้าง DCNN แบบ VGG 1 มิติ จำแนกสัตว์ 7 ประเภทจาก 16 features ได้สำเร็จ โดยทำ standardization และแบ่งข้อมูลแบบ stratified
- **จำนวน epoch:** 10 epochs ให้ test accuracy 0.875; ตั้งแต่ 25 epochs ขึ้นไปได้ 0.979–1.000; เลือก 50 epochs เพราะ validation loss ต่ำสุด

- **โครงสร้าง:** ทั้ง 9 แบบได้ test accuracy เฉลี่ย 0.958–1.000; โมเดล 6 ชั้นซึ่งใหญ่ที่สุดได้ผลต่ำสุด; เลือก 4 ชั้น convolution กับ 32 neurons (30,247 พารามิเตอร์)
- **โมเดลสุดท้าย:** train 100.00%, validation 93.75%, test 100.00% โดยต้องตีความร่วมกับข้อจำกัดเรื่องขนาดชุดข้อมูล

ภาคผนวก: วิธีรันซ้ำ

```
pip install -r requirements.txt
cd classification
python main.py            # ฝึกและทดลองทั้งหมด บันทึกผลลง outputs/ (ประมาณ 5–6 นาทีบน CPU)
python test_vgg.py        # ทดสอบโมเดลกับตัวอย่างสุ่ม 4 ตัว (ใส่ --seed 7 เพื่อให้ได้ผลตามรูปที่ 5)
```

โค้ดกำหนด random seed และเปิด deterministic ops ไว้ การรันซ้ำบนเครื่องเดิมจึงให้ตัวเลขเดียวกับในรายงานนี้ (ต่างเครื่องหรือต่างเวอร์ชัน TensorFlow อาจคลาดเคลื่อนได้เล็กน้อย)